UNIT-V: Agile Software Development

This unit covers several key areas within Agile Software Development, including an introduction to agile methods, agile development techniques, agile project management, and scaling agile methods. It also delves into Kanban principles, flow management, and continuous improvement, alongside the role of an Agile Coach, emphasizing understanding people's resistance to change, learning processes, and what makes a methodology effective.

Chapter 3 – Agile Software Development

Topics Covered

- Agile methods
- Agile development techniques
- Agile project management
- Scaling agile methods

Rapid Software Development

Rapid development and delivery are frequently the most critical requirements for contemporary software systems. Businesses operate in rapidly changing environments, making it practically impossible to establish a stable set of software requirements at the outset. Consequently, software must evolve quickly to reflect changing business needs. Plan-driven development, while essential for certain types of systems, typically does not fulfill these dynamic business requirements.

Agile Development

Agile development methods emerged in the late 1990s with the objective of significantly reducing the delivery time for functional software systems. In agile development:

- Program specification, design, and implementation activities are interleaved.
- The system is developed through a series of versions or increments.
- Stakeholders are actively involved in both the specification and evaluation of these versions.
- New versions are frequently delivered for evaluation.
- Extensive tool support, such as automated testing tools, is utilized to facilitate development.
- Documentation is minimal, with a strong focus on producing working code.

Plan-driven and Agile Development

Plan-driven Development A plan-driven approach to software engineering is structured around distinct development stages, where the outputs to be produced at each stage are planned in advance. This approach is not exclusively the waterfall model; plan-driven, incremental development is also possible. Iteration primarily occurs within individual activities.

Agile Development In contrast, agile development involves interleaving specification, design, implementation, and testing activities. The outputs from the development process are

determined through a continuous process of negotiation throughout the software development lifecycle.

Agile Methods

Agile methods arose from dissatisfaction with the extensive overheads associated with software design methods prevalent in the 1980s and 1990s. These methods are characterized by:

- A focus on the code rather than detailed design.
- An iterative approach to software development.
- The intention to deliver working software rapidly and to evolve it quickly to meet changing requirements. The primary aim of agile methods is to reduce process overheads, for example, by limiting documentation, and to enable swift responses to changing requirements without excessive rework.

Agile Manifesto

The Agile Manifesto articulates a set of values for uncovering better ways of developing software. It states a preference for:

- **Individuals and interactions** over processes and tools.
- **Working software** over comprehensive documentation.
- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a plan. While there is acknowledged value in the items on the right (processes and tools, comprehensive documentation, contract negotiation, following a plan), the items on the left are valued more.

The Principles of Agile Methods

Agile methods are guided by several core principles:

- **Customer Involvement** Customers should be closely engaged throughout the entire development process. Their role encompasses providing and prioritizing new system requirements, as well as evaluating the iterations of the system.
- **Incremental Delivery** The software is developed in increments, with the customer specifying the requirements to be included in each increment.
- **People Not Process** The skills of the development team should be recognized and effectively utilized. Team members should be empowered to develop their own working methods, rather than being constrained by prescriptive processes.
- **Embrace Change** System requirements are expected to change, and thus, the system should be designed to accommodate these changes.
- **Maintain Simplicity** Focus should be placed on simplicity in both the software being developed and the development process itself, whenever possible.

Agile Method Applicability

Agile methods are particularly suitable in the following contexts:

- **Product Development** Where a software company is developing a small or medium-sized product for commercial sale. Virtually all software products and applications are now developed using an agile approach.
- **Custom System Development within an Organization** When there is a clear commitment from the customer to be involved in the development process, and where there are minimal external rules and regulations affecting the software.

Agile Development Techniques

Extreme Programming (XP)

Extreme Programming (XP) is a highly influential agile method that emerged in the late 1990s, introducing a variety of agile development techniques. XP adopts an 'extreme' approach to iterative development. Its characteristics include:

- New versions may be built multiple times per day.
- Increments are delivered to customers every two weeks.
- All tests must be executed for every build, and the build is accepted only if all tests run successfully.

The Extreme Programming Release Cycle

Extreme Programming Practices

- **Incremental Planning** Requirements are recorded on 'story cards'. The stories to be included in a release are determined by the available time and their relative priority. Developers then break these stories down into development 'Tasks'.
- **Small Releases** The minimal useful set of functionality that provides business value is developed first. Subsequent releases of the system are frequent, incrementally adding functionality to the initial release.
- **Simple Design** Only sufficient design is carried out to meet the current requirements, and no more.
- **Test-first Development** An automated unit test framework is used to write tests for a new piece of functionality before the functionality itself is implemented.
- **Refactoring** All developers are expected to continuously refactor the code as soon as possible code improvements are identified.
- **Pair Programming** Developers work collaboratively in pairs, checking each other's work and providing mutual support to ensure high-quality output.
- **Collective Ownership** Pairs of developers work on all areas of the system, preventing the development of isolated areas of expertise. All developers take responsibility for the entire codebase, and anyone is permitted to modify any part of the code.
- **Continuous Integration** As soon as work on a task is completed, it is integrated into the overall system. Following any such integration, all unit tests within the system must pass.
- **Sustainable Pace** Large amounts of overtime are not considered acceptable, as their net effect often leads to reduced code quality and diminished medium-term productivity.
- **On-site Customer** A representative of the end-user (the customer) should be available full-time to the XP team. In an Extreme Programming process, the customer is an integral part of the team.

XP and Agile Principles

XP practices directly align with agile principles:

- **Incremental development** is supported through small, frequent system releases.
- **Customer involvement** is embodied by full-time customer engagement with the team.
- **People not process** is fostered through pair programming, collective ownership, and a process that discourages long working hours.
- **Change** is supported through regular system releases.
- **Maintaining simplicity** is achieved through constant refactoring of code.

Influential XP Practices

Extreme Programming has a strong technical focus, which can make its integration with management practices in most organizations challenging. As a result, while agile development commonly adopts practices from XP, the method as originally defined is not widely used. Key influential practices from XP include:

- User stories for specification.
- Refactoring.
- Test-first development.
- Pair programming.

User Stories for Requirements

In XP, a customer or user is an integral part of the XP team, responsible for making decisions regarding requirements. User requirements are expressed as user stories or scenarios, which are typically written on cards. The development team then breaks these stories down into implementation tasks, which serve as the basis for schedule and cost estimates. The customer selects the stories for inclusion in the next release based on their priorities and the schedule estimates. An example is a 'prescribing medication' story.

Refactoring

Conventional wisdom in software engineering advocates for designing for change, positing that investing time and effort in anticipating changes reduces costs later in the lifecycle. However, XP contends that this is not worthwhile because changes cannot be reliably anticipated. Instead, XP proposes constant code improvement, or refactoring, to facilitate changes when they eventually need to be implemented.

The programming team actively seeks out potential software improvements and implements them even when there is no immediate necessity. This practice enhances the understandability of the software, thereby reducing the need for extensive documentation. Changes become easier to implement because the code is well-structured and clear. However, it is important to note that some changes necessitate architecture refactoring, which can be significantly more expensive.

Examples of refactoring include:

- Reorganizing a class hierarchy to eliminate duplicate code.
- Tidying up and renaming attributes and methods to improve their clarity and understandability.
- Replacing inline code with calls to methods that have been incorporated into a program library.

Test-first Development

Testing is a fundamental aspect of XP, which has pioneered an approach where the program is tested after every modification.

XP testing features include:

- Test-first development.
- Incremental test development derived from scenarios.
- User involvement in test development and validation.
- Automated test harnesses are used to execute all component tests each time a new release is built.

Test-driven Development

Writing tests before writing code helps to clarify the requirements that need to be implemented. Tests are written as programs rather than just data, allowing for automatic execution. Each test includes a check to verify that it has executed correctly, often relying on a testing framework such as JUnit. All previously written tests, along with new ones, are automatically run when new functionality is added, ensuring that the new functionality has not introduced errors.

Customer Involvement in Testing

The customer's role in the testing process is to assist in developing acceptance tests for the user stories planned for the next system release. The customer, as part of the team, writes tests concurrently with development. This ensures that all new code is validated against what the customer genuinely needs. However, individuals in the customer role often have limited time, making full-time engagement with the development team challenging. They may also feel that providing requirements is a sufficient contribution and be reluctant to become further involved.

An example is a test case description for dose checking.

Test Automation

Test automation involves writing tests as executable components before the corresponding task is implemented. These testing components should be stand-alone, simulate input submission, and verify that the output meets specifications. An automated test framework, like JUnit, simplifies the process of writing executable tests and submitting a set of tests for execution.

Since testing is automated, there is always a readily available set of tests that can be executed quickly and easily. Whenever new functionality is added to the system, these tests can be run, and any problems introduced by the new code can be immediately identified.

Problems with Test-first Development

Despite its benefits, test-first development can encounter several issues:

- Programmers may prefer coding over testing and sometimes take shortcuts, writing incomplete tests that do not account for all possible exceptions.
- Certain tests can be very difficult to write incrementally. For instance, in complex user interfaces, writing unit tests for display logic and workflow between screens is often challenging.
- It can be difficult to assess the completeness of a set of tests. Even with a large number of system tests, the test suite may not provide comprehensive coverage.

Pair Programming

Pair programming involves two programmers working collaboratively to develop code. This practice fosters common ownership of code and disseminates knowledge across the team. It also functions as an informal review process, as each line of code is scrutinized by more than one person. Furthermore, it encourages refactoring, as the entire team can benefit from improvements to the system code.

In pair programming, programmers typically sit together at the same computer. Pairs are formed dynamically, ensuring that all team members work with each other throughout the development process. The knowledge sharing facilitated by pair programming is crucial, as it mitigates project risks when team members depart. Pair programming is not necessarily inefficient; some evidence suggests that a pair working together can be more efficient than two programmers working separately.

Agile Project Management

The primary responsibility of software project managers is to oversee the project to ensure software delivery on time and within budget. The conventional approach to project management is plan-driven, where managers create a plan detailing deliverables, timelines, and personnel assignments. Agile project management necessitates a different approach, adapted to incremental development and the specific practices used in agile methodologies.

Scrum

Scrum is an agile method that focuses on managing iterative development rather than prescribing specific agile practices. Scrum comprises three main phases:

- **Initial Phase**: An outline planning phase where general project objectives are established and the software architecture is designed.
- **Sprint Cycles**: This phase consists of a series of sprint cycles, each developing an increment of the system.

- **Project Closure Phase**: This final phase wraps up the project, completes necessary documentation (such as system help frames and user manuals), and assesses lessons learned from the project.

Scrum Terminology

- **Development Team** A self-organizing group of software developers, ideally no more than 7 people. They are responsible for developing the software and other essential project documents.
- **Potentially Shippable Product Increment** The software increment delivered from a sprint. The concept is that it should be 'potentially shippable,' meaning it is in a finished state requiring no further work, such as testing, to be integrated into the final product. In practice, this is not always achievable.
- **Product Backlog** A prioritized list of 'to-do' items that the Scrum team must address. These can include feature definitions, software requirements, user stories, or descriptions of supplementary tasks like architecture definition or user documentation.
- **Product Owner** An individual (or small group) tasked with identifying product features or requirements, prioritizing them for development, and continuously reviewing the product backlog to ensure the project meets critical business needs. The Product Owner can also be a customer representative.
- **Scrum** A daily meeting of the Scrum team to review progress and prioritize work for the day. Ideally, this is a short, face-to-face meeting involving the entire team.
- **ScrumMaster** Responsible for ensuring the Scrum process is followed and guiding the team in effective Scrum usage. The ScrumMaster interfaces with the rest of the company and protects the Scrum team from external interference. Scrum developers emphasize that the ScrumMaster is not a project manager, although others may find it difficult to distinguish the roles.
- **Sprint** A development iteration, typically lasting 2-4 weeks.
- **Velocity** An estimate of how much product backlog effort a team can complete in a single sprint. Understanding a team's velocity helps in estimating what can be covered in a sprint and provides a basis for measuring performance improvement.

The Scrum Sprint Cycle

Sprints are of a fixed duration, usually between 2 and 4 weeks. Planning for a sprint begins with the product backlog, which lists all work to be done on the project. The selection phase involves the entire project team collaborating with the customer to choose features and functionality from the product backlog to be developed during the sprint.

Once these are agreed upon, the team self-organizes to develop the software. During this stage, the team is shielded from direct interaction with the customer and the broader organization; all communications are channeled through the ScrumMaster. The ScrumMaster's role is to protect the development team from external distractions. At the conclusion of the sprint, the completed work is reviewed and presented to stakeholders, after which the next sprint cycle commences.

Teamwork in Scrum

The 'ScrumMaster' acts as a facilitator, organizing daily meetings, tracking the work backlog, recording decisions, measuring progress against the backlog, and communicating with

customers and management outside the team. The entire team participates in short daily meetings, known as Scrums, where all members share information, describe their progress since the last meeting, discuss any problems encountered, and outline plans for the following day. This ensures that everyone on the team is aware of ongoing activities and can re-plan short-term work to address emerging issues.

Scrum Benefits

Scrum offers several advantages:

- The product is broken down into manageable and understandable chunks.
- Unstable requirements do not impede progress.
- The entire team has visibility into all aspects of the project, which improves team communication.
- Customers receive on-time delivery of increments and gain feedback on how the product functions.
- Trust is established between customers and developers, fostering a positive culture where success is anticipated.

Scaling Agile Methods

Agile methods have proven successful for small to medium-sized projects that can be developed by a small, co-located team. It is sometimes argued that their success stems from enhanced communication facilitated by team proximity. Scaling up agile methods involves adapting them to larger, longer projects with multiple development teams, potentially working in distributed locations.

Scaling Up and Scaling Out

- **Scaling Up**: Concerns the application of agile methods for developing large software systems that cannot be handled by a small team.
- **Scaling Out**: Focuses on how agile methods can be introduced across a large organization with extensive software development experience.

When scaling agile methods, it is crucial to preserve fundamental agile principles: flexible planning, frequent system releases, continuous integration, test-driven development, and effective team communications.

Practical Problems with Agile Methods

Several practical challenges arise with agile methods:

- The informality inherent in agile development is often incompatible with the legalistic approach to contract definition commonly employed in large corporations.
- Agile methods are primarily suited for new software development, rather than software maintenance. However, the majority of software costs in large companies are attributed to maintaining existing systems.
- Agile methods are designed for small, co-located teams, yet much of modern software development involves globally distributed teams.

Contractual Issues

Most software contracts for custom systems are based on a detailed specification outlining what the system developer must implement for the customer. This approach, however, prevents the interleaving of specification and development, which is standard in agile development. A contract that compensates for developer time rather than delivered functionality is required. However, this is often perceived as high-risk by legal departments because the exact deliverables cannot be guaranteed.

Agile Methods and Software Maintenance

Most organizations allocate more resources to maintaining existing software than to developing new software. Therefore, for agile methods to be successful, they must support maintenance in addition to original development. Two key issues arise:

- Are systems developed using an agile approach maintainable, given the emphasis on minimizing formal documentation during development?
- Can agile methods be effectively used for evolving a system in response to customer change requests? Problems may also emerge if the original development team cannot be retained.

Agile Maintenance

Key problems in agile maintenance include:

- Lack of comprehensive product documentation.
- Difficulty in keeping customers continuously involved in the development process.
- Challenges in maintaining the continuity of the development team. Agile development heavily relies on the development team's intrinsic knowledge and understanding of what needs to be done. For systems with long lifespans, this presents a significant challenge, as the original developers will inevitably move on from the system.

Agile and Plan-driven Methods

Most software projects incorporate elements from both plan-driven and agile processes. The appropriate balance depends on several factors:

- Is it crucial to have a highly detailed specification and design before proceeding to implementation? If so, a plan-driven approach is likely necessary.
- Is an incremental delivery strategy, involving rapid software delivery to customers and immediate feedback, realistic? If so, agile methods should be considered.
- What is the scale of the system being developed? Agile methods are most effective when a system can be developed by a small, co-located team capable of informal communication. For large systems requiring larger development teams, this may not be feasible, potentially necessitating a plan-driven approach.

Agile Principles and Organizational Practice

Principle	Practice

Customer involvement	Requires a customer who is willing and able to dedicate time to the development team and who can represent all system stakeholders. Often, customer representatives have competing demands on their time and cannot fully participate. Representing the views of external stakeholders, such as regulators, to an agile team can also be challenging.
Embrace change	Prioritizing changes can be exceptionally difficult, especially in systems with numerous stakeholders, as each stakeholder often assigns different priorities to various changes.
Incremental delivery	Rapid iterations and short-term development planning may not align with the longer-term planning cycles common in business and marketing. Marketing managers, for instance, may require knowledge of product features several months in advance to prepare an effective marketing campaign.
Maintain simplicity	Under pressure from delivery schedules, team members may not have the time to implement desirable system simplifications.
People not process	Individual team members may not possess suitable personalities for the intense involvement typical of agile methods, potentially leading to poor interactions with other team members.

Agile and Plan-based Factors

System Issues

- **System Size**: Agile methods are most effective for systems that can be developed by a relatively small, co-located team capable of informal communication.
- **System Type**: Systems requiring extensive analysis before implementation generally necessitate a fairly detailed design to conduct this analysis effectively.
- **Expected System Lifetime**: Long-lifetime systems require documentation to convey the developers' intentions to the support team.
- **External Regulation**: If a system is subject to regulation, detailed documentation, often as part of system safety requirements, will likely be mandated.

People and Teams

- **Developer Skill Levels**: It is sometimes argued that agile methods demand higher skill levels from designers and programmers compared to plan-based approaches, where programmers might simply translate a detailed design into code.
- **Team Organization**: Design documents may become necessary if the development team is distributed.
- **Support Technologies**: Integrated Development Environment (IDE) support for visualization and program analysis is essential when design documentation is not available.

Organizational Issues

- Traditional engineering organizations often possess a culture rooted in plan-based development, as this is the industry norm.
- Is it standard organizational practice to develop a detailed system specification?
- Will customer representatives be available to provide feedback on system increments?

- Can informal agile development practices integrate with an organizational culture that emphasizes detailed documentation?

Agile Methods for Large Systems

Large systems are typically collections of separate, communicating systems, with individual teams often developing each system. Frequently, these teams are geographically dispersed, sometimes across different time zones. Large systems are often 'brownfield systems,' meaning they incorporate and interact with numerous existing systems. Many requirements for such systems revolve around these interactions, making them less amenable to flexibility and incremental development. When multiple systems are integrated to form a larger system, a significant portion of development involves system configuration rather than original code development.

Large System Development Considerations

- Large systems and their development processes are frequently constrained by external rules and regulations, dictating how they can be developed.
- Large systems typically have long procurement and development times, making it difficult to maintain cohesive teams with system knowledge over extended periods, as personnel inevitably transition to other roles or projects.
- Large systems usually involve a diverse array of stakeholders, making it practically impossible to involve all of them in the development process.

Scaling Up to Large Systems

When scaling agile methods for large systems:

- A completely incremental approach to requirements engineering becomes impossible.
- There cannot be a single product owner or customer representative.
- It is not feasible to focus solely on the system's code.
- Specific cross-team communication mechanisms must be designed and implemented.
- Continuous integration, as practiced in smaller teams, is practically impossible. However, maintaining frequent system builds and regular releases of the system remains essential.

Multi-team Scrum

For large-scale Scrum implementations involving multiple teams:

- **Role Replication**: Each team is assigned a Product Owner for its specific work component and a ScrumMaster.
- **Product Architects**: Each team selects a product architect, and these architects collaborate to design and evolve the overall system architecture.
- **Release Alignment**: The release dates from each team are synchronized to ensure that a demonstrable and complete system is produced.
- **Scrum of Scrums**: A daily 'Scrum of Scrums' meeting is held, involving representatives from each team.

Agile Methods Across Organizations

Implementing agile methods in large organizations can face resistance:

- Project managers lacking experience with agile methods may be reluctant to accept the perceived risks of a new approach.
- Large organizations often have quality procedures and standards that all projects must follow. Due to their bureaucratic nature, these are likely to be incompatible with agile methods.
- Agile methods generally perform best when team members possess relatively high skill levels. However, large organizations often have a wide range of skills and abilities among their staff.
- Cultural resistance to agile methods may exist, particularly in organizations with a long history of traditional development practices.

Key Points

- Agile methods are incremental development approaches focused on rapid software development, frequent software releases, reduction of process overheads by minimizing documentation, and producing high-quality code.
- Key agile development practices include user stories for system specification, frequent software releases, continuous software improvement, test-first development, and customer participation in the development team.
- Scrum is an agile method that provides a project management framework, centered around a series of 'sprints'—fixed time periods during which a system increment is developed.
- Many practical development methods are a hybrid of plan-based and agile approaches.
- Scaling agile methods for large systems presents challenges, as large systems typically require up-front design and some documentation. Additionally, existing organizational practices may conflict with the informality characteristic of agile approaches.

CHAPTER 9: Kanban, Flow, and Constantly Improving

Introduction to Kanban

- **Kanban** is a method for **process improvement** used by agile teams.
- It is **not** a software development lifecycle methodology or an approach to project management.
- Kanban requires an existing process to be in place, which it then incrementally changes.
- Teams using Kanban begin by understanding their current software building process and improve it over time.
- The Kanban method necessitates the **lean thinking mindset**.
- Lean is a mindset characterized by specific values and principles.
- Teams applying Kanban use the Lean mindset as a foundation to improve their process.
- When improving with Kanban, teams focus on **eliminating waste**.
- Waste includes *muda* (unevenness), *mura* (overburdening), and *muri* (futility).
- **Origin and Relationship to Lean:**

    - Kanban is a manufacturing term adapted for software development by **David Anderson**.
    - Anderson describes the Kanban Method as a complex adaptive system intended to catalyze a Lean outcome within an organization.
    - While Lean and lean thinking can be applied without Kanban, Kanban is widely considered the most common and effective way to bring lean thinking into an organization by many agile practitioners.
- **Distinction from other Agile Methodologies:**
    - Kanban has a different focus compared to agile methodologies like Scrum and XP.
    - **Scrum** primarily focuses on **project management**, including scope, delivery timelines, and meeting user/stakeholder needs.
    - **XP (Extreme Programming)** focuses on **software development**, with values and practices aimed at creating an environment conducive to development and fostering programmer habits for simple, changeable code.
    - **Kanban** is centered on helping a team **improve how they build software**.
- **Benefits of Kanban:**
    - A team using Kanban gains a clear understanding of their software building actions, company interactions, sources of waste (inefficiency, unevenness), and how to improve by removing root causes.
    - This improvement process is traditionally called **process improvement**.
    - Kanban applies agile ideas, like the "last responsible moment," to create a straightforward and easily adoptable process improvement method.

Core Principles and Practices of Kanban

- Kanban includes practices to stabilize and improve a software building system.
- The latest core practices are available via the Kanban Yahoo! group.
- **Foundational Principles:**
    - **Start with what you do now**: Do not change the existing process immediately.
    - **Agree to pursue incremental, evolutionary change**: Changes are small and iterative.
    - **Initially, respect current roles, responsibilities & job titles**: Do not impose new roles at the outset.
- **Core Practices:**
    - **Visualize**: Make the workflow visible.
    - **Limit WIP (Work In Progress)**: Restrict the amount of work in each stage.
    - **Manage Flow**: Optimize the movement of work items.
    - **Make Process Policies Explicit**: Clearly define the rules and guidelines.
    - **Implement Feedback Loops**: Create mechanisms for continuous learning and adjustment.
    - **Improve Collaboratively, Evolve Experimentally (using models/scientific method)**: Foster a culture of team-driven, data-backed improvement.
- It is not expected that all six practices are adopted initially; partial implementations are termed "shallow" with an expectation of gradual depth increase.
- **Chapter Focus:** This chapter covers Kanban's principles, its relation to Lean, its practices, emphasis on flow and queuing theory, and its role in fostering continuous improvement.

Narrative Example: Playing Catch-Up

- **Scenario:** Catherine and Timothy (developers) are frustrated with their boss, Dan, who constantly micromanages them, calling it "Intensive Care Unit mode".
- **Problem:** Their current project, a phone camera app feature, is experiencing bugs during testing, leading to missed deadlines.
- **Dan's Response:** Dan attributes delays to the developers, stating projects are "falling apart" and lacking urgency, insisting on "the right way". He dismisses "waste" as negative.
- **Team's Perspective:**
    - Catherine identifies waste, such as weeks-long UI agreement processes leading to extensive code changes.
    - Timothy points out the recurring surprise of QA finding bugs, with insufficient time allocated for fixes.
    - Catherine argues that problems are habitual and repeatedly occur, requiring a different approach than just "more urgency".
- **Outcome of Discussion:** Dan insists on more "time and urgency," maintaining it's "crunch time". This illustrates a common problem where teams are blamed for systemic issues.

The Principles of Kanban (Detailed)

- **"Start with what you do now"**:
    - This principle contrasts with agile methodologies like Scrum, which provide a complete system requiring new roles (Scrum Master, Product Owner) and activities (sprint planning, Daily Scrum, task boards) for adoption.
    - Kanban is **not** a project management system; it is a method for **improving your process**.
    - The starting point for Kanban improvement is the team's current way of building and delivering software.
- **Finding a Starting Point and Evolving Experimentally:**
    - Habitual problems on a project, such as those leading to bugs and missed deadlines, often don't feel like mistakes at the time, and teams tend to repeat them.
    - Example: A programming team repeatedly delivers software where users can't find promised features, indicating a recurring problem with requirements gathering or communication, rather than developer forgetfulness.
    - The **goal of process improvement** is to identify recurring problems, find commonalities, and develop tools to fix them.
    - Assuming a fixable root cause, rather than attributing problems to individual incompetence or external factors, is crucial for finding solutions.
    - Kanban begins by viewing current work as a set of changeable, repeatable steps.
    - **Policies** are steps or rules that Kanban teams always follow.
    - This involves teams recognizing their habits, documenting their recurring steps in software building.
    - Judging teams solely on project results is unfair, as it ignores the larger system at play; Lean thinking emphasizes "seeing the whole".
- **Every Team Has a System:**

    - Every team possesses a system for building software, even if it's chaotic, changes frequently, or exists only as "tribal knowledge" (e.g., starting projects by talking to specific customer representatives or immediate coding after a quick manager meeting).
    - Methodologies like Scrum codify their systems.
    - Kanban starts with this existing system, asking teams to understand it.
    - The goal is to make **small improvements** to this system, aligning with the "pursue incremental, evolutionary change" principle and the "improve collaboratively, evolve experimentally" practice.
    - Lean thinking encourages **measurements** as core to experimentation and the scientific method.
    - Kanban teams objectively understand their system through measurements, make specific team changes, and then re-measure to evaluate effects.
    - **Amplifying learning** (a Lean value) is vital for evolving the system, with **feedback loops** (a Kanban practice) serving as crucial tools for gathering and integrating information.
- **"Initially, respect current roles, responsibilities, and job titles"**:
    - This principle ties into amplifying learning.
    - Existing roles (e.g., project manager, business analyst, programmer meeting) are important parts of the system and provide insight into unwritten rules.
    - **Note**: Kanban is not a project management method, but it is beneficial for project managers.
- **The "No Silver Bullet" Principle:**
    - Kanban's principles collectively emphasize that teams must understand their own system for building software.
    - There is no single "best" set of practices guaranteeing success; even the same team can have varying results across projects.
    - Kanban begins with understanding the current project system to then provide practices for improvement.
- **Kanban's Purpose (Clarified):**
    - Kanban doesn't dictate *how* to run a project; instead, it helps you understand your current project execution (whether Scrum, XP, waterfall, or haphazard) and offers practices for improvement.
    - Teams need Kanban to identify wasted time/effort, ensure value delivery, and address habitual problems that make software delivery difficult.
    - Habitual problems and waste can emerge from the way many people work together, even if everyone follows rules.
    - Example: A team experiencing long lead times despite constant individual work, due to systemic issues rather than individual slacking. Kanban addresses such problems.
- **Systems Thinking:**
    - "Systems thinking looks at organizations as systems; it analyzes how the parts of an organization interrelate and how the organization as a whole performs over time." —Tom and Mary Poppendieck.
    - Recognizing the existence of a system is the first step in improving it, aligned with the Lean principle of "seeing the whole".
    - Systems thinking views the team as following a system rather than making disconnected decisions.
    - Example: A Scrum team can be seen as a system that takes project backlog items (e.g., stories) as input and produces code as output.

    - This perspective helps identify work that doesn't directly contribute to converting stories into code.
    - Applying systems thinking to Scrum, as seen in Jeff Sutherland's Daily Scrum question changes, leads to incremental, evolutionary improvements.
    - Kanban encourages understanding your team's system, even if no formal methodology is followed.
- **Every Software Team Follows a System (Written or Unwritten):**
    - Humans naturally follow rules, even unwritten ones, and intuit them if they don't exist.
    - A **value stream map** is a Lean tool to transform unwritten rules into a system.
    - Mapping the path of a Minimal Marketable Feature (MMF) from request to code creates a description of a system path.
    - Even with many unwritten rules, a small number of value streams likely cover the majority of MMFs.
    - The first step in such a system is deciding which value stream the MMF will follow.
    - Having a consistent system is valuable, and understanding the whole system allows for informed decisions about wasteful paths and incremental, evolutionary changes.
- **Kanban is NOT a Software Methodology or Project Management System (Reiteration):**
    - It is a method for **process improvement**, helping teams understand and improve their existing methodologies.
- **Kanban Boards vs. Task Boards:**
    - **Kanban boards** are tools used by teams to visualize their workflow.
    - They typically consist of columns on a whiteboard with sticky notes.
    - **Key Differences:**
        1. **Work Items vs. Tasks**: Kanban boards feature **work items** (single, self-contained units of work tracked through the entire system, typically larger than MMFs, requirements, or user stories), **not tasks**. Tasks are what people do to move work items.
        2. **Column Variability**: Columns on kanban boards often **vary from team to team**, unlike more standardized task boards.
        3. **WIP Limits**: Kanban boards can set **limits on work in progress** in a column, a feature not typically found on task boards.
    - **Workflow Mapping**: Kanban practitioners distinguish workflow mapping (determining actual steps a work item goes through) from value stream mapping (a Lean tool for understanding the system).
- **Kanban Board in Practice (Example):**
    - David Anderson's book *Kanban* features an example board with columns: Input Queue, Analysis (In Prog), Analysis (Done), Dev Ready, Development (In Prog), Development (Done), Build Ready, Test, and Release Ready.
    - Teams use kanban boards similarly to Scrum task boards, holding daily "walking the board" meetings to discuss and update the state of each item.
    - **Crucial Note**: A kanban board visualizes the *underlying workflow* in use; teams should **never copy** another kanban board but develop their own by studying their workflow, as copying contradicts Kanban's evolutionary approach.
- **Visualizing Problems with Kanban Boards (Example from Chapter 8):**

    - Initial workflow (project manager's "happy path"): Feature request -> Schedule -> Build -> Test -> Verify -> Done/Release.
    - Real-life workflow (after Five Whys): Adds a **Manager Review** step where features can be sent back to step 1 for changes if senior management desires.
    - The kanban board is modified to include a "Manager Review" column, accurately reflecting this step.
    - **Problem Visualization**: Work items piling up in "Manager Review" indicates unevenness (*mura*).
    - Adding a "Bumped by Managers" column for items returned to the beginning of the process explicitly visualizes the waste of rework.
    - This makes waste objective and explicit, showing the root cause of lead time problems (senior manager review process).

Improving Your Process with Kanban

- Kanban is an agile method focused on process improvement, based on Lean values and thinking, formulated by David Anderson.
- Traditional process improvement often involves senior management sponsorship, committee decisions, external consultants, and extensive documentation.
- This frequently leads to developers' frustration, awkward processes, and superficial compliance rather than genuine adoption.
- **Key Difference (Kanban vs. Traditional Process Improvement)**:
    - In traditional approaches, decisions are top-down.
    - In Kanban, **improvement is in the hands of the team**: team members identify problems, suggest improvements, measure results, and hold themselves accountable. This is a reason for Kanban's success with agile teams.

Visualize the Workflow

- The first step in process improvement is understanding the current team workflow, which is the aim of the **Visualize** practice.
- It's challenging to accurately visualize without embellishment; a common mistake is adding steps that *should* happen but don't always (e.g., code reviews).
- Such embellishment masks real problems and prevents addressing underlying issues (e.g., senior team members always being too busy for code reviews).
- In Kanban, visualizing means writing down **exactly what the team does, warts and all, without embellishing it**.
- This embodies the Lean principle of "see the whole" and "decide as late as possible" (as information about how to change is not yet complete).
- Accurately visualizing the workflow helps embed Lean values like "see the whole" and "decide as late as possible" into team thinking.
- **Using a Kanban Board to Visualize Workflow:**
    - A kanban board is a tool for workflow visualization (Kanban is capitalized for the method, kanban board is lowercase).
    - It resembles a Scrum task board but uses sticky notes (more common than index cards).
    - **Key Differences Recap**:
        - Kanban boards show **work items**, not individual tasks.
        - Columns vary by team.
        - Kanban boards can have **WIP limits**.

Limit Work in Progress (WIP)

- **Concept**: A team has a finite capacity for work at any given time.
- **Consequences of Overburdening**: Agreeing to do more work than can be accomplished by a deadline leads to:
    - Leaving work out of delivery.
    - Poor product quality.
    - Unsustainable pace (future costs).
    - Multitasking across multiple projects/tasks can lead to overburdening and task-switching overhead that consumes up to half of productivity.
- **Visualizing Overburdening**: Visualizing the workflow helps identify overburdening when sticky notes consistently pile up in one column.
- **Solution**: Queuing theory suggests fixing unevenness by placing a strict limit on the amount of work allowed to pile up. This is the **Limit Work in Progress** practice.
- **WIP Limit Definition**: Setting a limit on the number of work items allowed in a particular stage of the workflow.
- **Focus Shift**: Instead of fixating on quickly moving a single work item (e.g., development to testing), Kanban encourages considering the whole system.
    - If the test team is overburdened, starting new development for a feature that will just wait is inefficient.
- **Options Thinking**: Kanban boards reveal all available options, aligning with the Lean principle of options thinking.
    - Completing a design doesn't commit a developer to coding next; other options might include designing other features or fixing bugs in later columns.
- **Impact of WIP Limits**: Limiting WIP for a step restricts options, making decisions easier, preventing overburdening, and promoting efficient flow.
    - Example: If the code writing workflow hits its WIP limit, a developer will choose other work, preventing the test team from being overburdened.
    - This reduces the average lead time for a feature.
- **Implementing WIP Limits (Case Study Continuation):**
    - For the team with "Manager Review" logjams, a WIP limit is set to control the number of features accumulating before manager review.
    - Objective evidence from the kanban board and lead time measurements helps convince managers to agree to a new policy.
    - A WIP limit transforms a "sinkhole" column into a **queue**, managing work items smoothly.
    - **Determining WIP Limit**: No hard-and-fast rule; teams take an evolutionary approach, choosing a sensible limit, then experimentally adjusting it based on measurements.
    - Example: For 30 features per release, with managers meeting three times, a WIP limit of 10 for "Manager Review" is chosen. This is visually added to the kanban board.
- **Effect of Reaching WIP Limit:**
    - When a column hits its WIP limit, no more work items are pushed into it.
    - The team shifts focus to other work items or technical debt not at their WIP limits.
    - The agreement with managers is that hitting the WIP limit prompts a review meeting.
    - This forces managers to provide feedback earlier, allowing the team to make immediate adjustments.

    - Early feedback means managers know about potential feature bumps sooner, making it less disruptive and preventing valuable work from becoming stale.
- **WIP Limits and Feedback Loops:**
    - The cycle of manager review and team adjustment forms a **feedback loop**.
    - A too-long feedback loop (e.g., entire release length) causes disruptive information and waste (shelving completed work).
    - Adding a WIP limit shortens average work item time in the system, as managers review more frequently, allowing earlier adjustments and priority changes.
    - Teams and managers gain control over feedback loop length; e.g., shrinking WIP limit to 6 features for more frequent feedback.
    - **Thrashing**: A system can thrash if a feedback loop is too short, leading to too much information without enough time to respond.
    - Visualizing with a kanban board helps identify feedback loops and experiment with WIP limits to find an optimal length.
    - Avoiding repeated feedback loops (items repeatedly sent back to earlier stages) prevents clogging the system.
        - Marking stickies moved backward (e.g., with a dot) indicates repeated feedback loops and potential clogging.
    - **Optimized Workflow Example**: Convincing managers to limit features bumped back to backlog, and adding an extra development/testing step post-review, makes the workflow more linear for most features.
    - This "longer" workflow can actually be faster due to reduced unevenness and overburdening, objectively proven by lead time measurements and visualization. The team feels the improved speed.

Key Points (Summary of Kanban Basics)

- **Kanban** is a **process improvement method** based on the **Lean mindset**, helping teams improve how they build software.
- **Starting Point**: Kanban teams **start with what they do now**, "seeing the whole" current system, and pursuing **incremental, evolutionary change**.
- **Improve Collaboratively, Evolve Experimentally**: This practice involves taking measurements, making gradual improvements, and using measurements to confirm their effectiveness.
- **Systems Thinking**: Every team has a software building system, and Lean's systems thinking helps teams understand it.
- **Kanban Board**: A tool for Kanban teams to **visualize their workflow**.
    - Columns represent workflow stages, and stickies represent work items flowing through the system.
    - Kanban boards use **work items, not tasks**, because Kanban is not a project management system.
- **Limit WIP (Work In Progress)**: Teams add a number to a column on the kanban board to indicate the maximum number of work items allowed in that stage.
- **Policy Agreement**: Kanban teams work with stakeholders to ensure agreement that when a column is at its WIP limit, the team will focus elsewhere, not advancing more work items into that stage.

Measure and Manage Flow

- **Flow Definition**: The rate at which work items move through the system.
- **Maximizing Flow**: Achieved when teams reach an optimal delivery pace combined with a comfortable amount of feedback.
- **Impact of Flow**:
    - Increased flow comes from cutting down unevenness and overburdening, allowing teams to finish tasks and move on.
    - Piling up work due to unevenness interrupts work and decreases flow.
    - The **Manage Flow** practice involves measuring flow and actively improving it.
- **Subjective Experience of Flow**:
    - **High Flow**: Feeling productive, not wasting time, not waiting, doing valuable work daily.
    - **Low Flow**: Feeling stuck, slow progress, constantly waiting for others (decisions, approvals, work completion), uncoordinated, spending time explaining delays. Project plan might show high completion, but it feels far from done. Users experience increasing lead times.
- **Kanban's Objective**: To increase flow and involve the entire team in this effort, which reduces frustration from unevenness and long lead times.
- **WIP Limits and Flow**: The kanban board visualizes flow problems, and WIP limits focus team effort on bottlenecks, thereby increasing flow. WIP limits change what work items the team focuses on, helping clear unevenness.

Use CFDs and WIP Area Charts to Measure and Manage Flow

- **Cumulative Flow Diagram (CFD)**: An effective tool for measuring flow, based on Lean thinking's emphasis on measurements.
- **Relationship to WIP Area Chart**: A CFD is similar to a WIP area chart, but work items accumulate in the final stripe/bar, rather than flowing off.
- **Stripe Correspondence**: CFDs (and WIP area charts in this context) have stripes corresponding to columns on a kanban board.
- **CFD Data Points**: Can show:
    - **Average arrival rate (λ)**: Number of work items added to the workflow daily.
    - **Average inventory (L)**: Total number of work items in the workflow.
    - **Average lead time (W)**: Amount of time each work item stays in the system.
    - Not all CFDs include all these lines, but they are helpful for understanding flow.
- **Managing Flow with CFD**: Look for patterns indicating problems.
    - Kanban boards help manage flow daily by showing unevenness and enabling WIP limits.
    - CFDs allow long-term analysis of process performance to find and fix root causes of long-term problems.
- **Building a Cumulative Flow Diagram:**
    - Start with a WIP area chart.
    - Gather data from the number of work items in each kanban board column.
    - Add daily arrival rate (count items added to first column) and inventory (total items in all columns) as dots, connecting them to form line charts.
    - Most teams use spreadsheets (e.g., Excel) for CFDs due to data management and automatic trendlines.
    - **Trendlines**: Linear trendlines for arrival rate and inventory indicate system stability.

        - Flat/horizontal: Stable system.
        - Tilted: Value is changing over time, indicating instability.
        - WIP limits are needed to stabilize; flatness indicates stability.
- **Interpreting a CFD (Example):**
    - Figure 9-10 (not provided here, but described): Shows L (average long-term inventory), λ (average arrival rate), and W (average lead time).
    - If inventory is trending downward and arrival rate increasing, it implies more items are leaving than arriving. If more features arrive than leave, inventory trends upward, causing stress and feeling "mired in muck".
    - **Solution**: Add WIP limits to balance arrival and completion rates, leading to flat long-term inventory and arrival rates, signifying a stable system.
- **Little's Law**:
    - A theorem from queueing theory by John Little, applicable to stable systems.
    - **Formula**: $\mathbf{L = W \times \lambda}$.
        - **L**: Average inventory.
        - **W**: Average lead time (average time a user waits for work item completion).
        - **λ**: Average arrival rate (number of work items added daily).
    - This is a mathematical law; if a system is stable, it is always true.
    - **Rearranged Formula**: $\mathbf{W = L \div \lambda}$.
        - If average inventory and arrival rate are known, average lead time can be calculated.
        - Daily work item counts from the kanban board (total and new items in first column) provide this data.
        - Once stable, average daily inventory and arrival rate (thick straight lines on CFD) can be used.
- **Significance of Lead Time and Quality:**
    - Lead time is a key measure of user frustration (shorter = pleased, longer = frustrated).
    - Long lead times indicate quality problems; David Anderson noted a 6.5x increase in average lead time resulted in over 30x increase in initial defects.
    - Longer lead times stem from greater WIP.
    - **Management Leverage Point for Quality**: Reduce the quantity of work in progress.
    - Lead time is determined by arrival rate and flow rate, with WIP limits controlling flow.
    - In a stable system, lead time can be reduced by **not starting work on new features**.
- **Using CFD to Experiment with WIP Limits and Manage Flow:**
    - Kanban's core idea: visualize workflow, measure flow, stabilize the system, and control lead time by managing the rate of new work item starts.
    - **Doctor's Office Example (Analogy):**
        - **Problem**: Long patient wait times, especially later in the day, leading to rushed doctors.
        - **Visualization**: Kanban board with columns: Waiting Room, Exam Rooms, Seeing Doctor. Natural WIP limits exist (5 exam rooms, 2 doctors).
        - **Data Collection**: Office staff counts patient arrivals (arrival rate) and total patients in waiting/exam rooms (inventory) every 15 minutes. Stickies on the board track patient flow.

        - **CFD Analysis**: Initial CFD shows stable arrival rate (due to appointment booking) but upward-tilted inventory (waiting room fills throughout the day), indicating instability.
        - **Stabilization with WIP Limit**: Office staff decides to add a WIP limit of six to the waiting room.
        - **Explicit Policy**: They agree that if the waiting room hits 6 patients, low-priority appointments for the next hour are rescheduled (with priority for reschedule). This policy is explicitly posted.
        - **Class of Service**: They define pink stickies for severe conditions (non-reschedulable) and yellow for minor problems to prioritize.
        - **Result**: The new policy works, allowing later scheduling, reducing patient waiting times, and providing better care.
- **Little's Law and Flow Control:**
    - The office staff discovers that in a stable system, they can control patient waiting time by adjusting the arrival rate.
    - **Example Calculation**:
        - L = 7 patients, λ = 11/hour -> W = 7/11 = 0.63 hours = 37 minutes.
        - L = 4 patients, λ = 10/hour -> W = 4/10 = 0.4 hours = 24 minutes.
    - Reducing scheduled patients by one per hour (from 11 to 10), and subsequent inventory drop (from 7 to 4), reduced waiting time by almost 15 minutes.
    - **Mechanism**: WIP limits reduce unevenness, preventing inventory buildup. This allows lead time reduction by reducing arrival rate (e.g., backlogging work or scheduling fewer patients).
    - Little's Law applies to *every stable system*, regardless of awareness.
- **Production Support Example (Software Team):**
    - **Problem**: Team feels constant stress and insufficient time due to periodic spikes in production support work every three weeks, leading to "sinking in quicksand" feeling and technical debt.
    - **CFD/WIP Area Chart Analysis**: A WIP area chart (similar to CFD, but showing work "flowing off") reveals that a specific stripe (column) gets thicker after each release, indicating work accumulation and an unstable system.
    - **Solution**: Adding a WIP limit to the accumulating column.
    - **Effect**: Spikes remain, but total tasks don't increase. The queue prevents extra work from flowing into the system, increasing overall flow and distributing work more evenly.
    - When the overburdened column hits its WIP limit, the team shifts focus to earlier columns, making those bumps smaller.
    - **Continuous Improvement**: Kanban teams continuously adjust WIP limits through feedback loops and experimentation, forming hypotheses and using measurements to prove them. This embodies "improve collaboratively and evolve experimentally".
    - **Benefits**: Increased flow leads to better focus, improved quality (less technical debt), and a more energized work environment.
    - **Addressing the "Extra Work" Question**: The extra production support work is no longer accumulating because it's not being *added* to the project in the first place. The WIP limit forces the boss to make a choice between support and development, rather than magical thinking that both can be handled.
    - If support work fills the queue, it explicitly prioritizes support over development.

    - **Impact on Management**: The boss now receives more accurate progress information (less software delivered if support is prioritized). This eliminates magical thinking and prompts decisions about resource allocation (prioritize or hire more people).

Managing Flow with WIP Limits Naturally Creates Slack

- **Slack Definition**: "Wiggle room" in the schedule for developers to do quality work. Lack of slack leads to corner-cutting and technical debt.
- **Impact of Rushed Environment**: Stifles creativity, innovation, and quality practices (e.g., cutting activities not directly producing code). XP teams include Slack as a primary practice.
- **Kanban's Value of Slack**: Teams using Kanban value slack and limit WIP to create it.
- **Delivery Cadence**: Instead of strict timeboxes with fixed scope, Kanban teams adopt a delivery cadence (e.g., release every six weeks) but don't commit to a specific set of work items. They trust the system, once unevenness and overburdening are removed, to deliver completed work items naturally.
- **Feeling of Relief**: Many people experience relief when WIP limits are agreed upon.
    - Before WIP limits: Teams relied on slack to absorb last-minute changes and emergency requests.
    - After WIP limits (and adherence): New work goes into a queue, removing the pressure of unlimited work.
    - Managed flow (possibly with CFD) ensures the queue's WIP limit provides enough time.
- **WIP Limits on Every Column**: Many Kanban teams set WIP limits on all columns of the kanban board to maximize flow throughout the entire project.
    - This helps control flow through each development step, even "Ready for Release".
    - Too much "done" work ready for release provides information to adjust future delivery cadence (e.g., reduce time between releases).
- **Collaborative and Experimental Evolution**: Teams may not set all WIP limits initially but gain manager agreement for more as improvements are demonstrated.
- **Outcome**: Controlling flow across the project helps teams relax, focus on current work, and trust WIP limits to manage chaos. Measurements help adjust limits and cadence if chaos appears.

Make Process Policies Explicit So Everyone Is on the Same Page

- **Explicit Policies (Scrum vs. Waterfall)**:
    - Effective Scrum teams have a common understanding of their rules, leading to consistent descriptions of how they build software.
    - Ineffective waterfall teams often have fractured perspectives, with individuals describing only their own work, or project managers describing planning/tracking steps.
- **Purpose of Explicit Policies**:
    - To determine if a complex process is necessary or wasteful bureaucracy.
    - Example: An unwritten rule of sending many emails for specification updates might be revealed as unnecessary bureaucracy once made explicit.
    - A project manager might justify a complex process for regulatory compliance.

- **Kanban Implementation of Explicit Policies**:
    - Does not require long documents or large Wikis.
    - Can be as simple as **WIP limits at the top of columns**.
    - Can include "definitions of done" or "exit criteria" as bullets at the bottom of columns.
    - Most effective when established collaboratively and evolved experimentally, fostering understanding of *why* policies exist.
- **Root Cause of Complex Processes**: Complex processes and unwritten rules often emerge defensively over time (e.g., a business analyst imposing complex change control to manage scope and avoid blame for last-minute changes).
- **WIP Limits as Policy Choice**: Setting WIP limits is a policy choice that works only if everyone honors the agreement not to push more work onto a full queue.
    - Writing down this agreement (especially visibly on a kanban board) reinforces it.
    - It gives the team leverage when managers or users try to push additional work, as they can point to the agreed-upon explicit policy.

Emergent Behavior with Kanban

- **Critique of Command-and-Control**: Micromanaging every activity (e.g., doctors estimating time per patient) is stifling and ineffective. Doctors are trained in medicine, not estimation, leading to potentially unrealistic schedules.
- **Kanban's Aggregate Approach**: Kanban uses **systems thinking** to understand, measure, and incrementally improve the system as a whole. It accepts individual work item variation but predicts system behavior when *muda*, *mura*, and *muri* (unevenness, overburdening, futility) are reduced.
- **Organizational Change**: As teams improve their process with Kanban, behavior in the rest of the company often changes.
    - Example: Lead time reduction in the previous case study.
    - **Mechanism**: Visualizing the workflow with a kanban board helps unfracture perspectives, allowing managers to see work piling up and agree to WIP limits and more frequent reviews. This is how WIP limits change behavior.
- **Conflict Resolution Example**:
    - **Problem**: Team receives conflicting requests from multiple managers, each believing their request is most important, leading to pressure and impossible choices.
    - **Kanban Solution**: Visualizing the workflow puts all manager requests (work items) in the first column. If more requests than capacity, stickies pile up, making the problem visible.
    - The team can then get managers to agree to a WIP limit on that column.
    - **Behavioral Change**: When a manager hits the WIP limit, they cannot simply add a new sticky; they must choose another to remove, potentially requiring negotiation with other managers.
    - **Systems Thinking in Action**: The manager recognizes the WIP limit as a system limitation, not a team blame point, and collaborates with others. Overburdening becomes *everyone's* problem, not just the team's.
    - **Outcome**: New behaviors emerge outside the team (e.g., prioritization meetings, "horse trading" among managers). The team is no longer blamed for overcommitment and gains the slack needed to work effectively.

    - **Eliminating Magical Thinking**: The WIP limit and explicit policy eliminate the "magical thinking" of expecting unlimited work from the team. Managers change behavior naturally due to the system, leading to more accurate progress information and less blame for the team.

Key Points (Summary of Flow Management and Emergent Behavior)

- **Maximizing Flow**: A primary goal of a Kanban team is to maximize the rate at which work items move through the system.
- **Measure and Manage Flow**: This practice involves taking flow measurements and adjusting the process to achieve maximum flow.
- **Cumulative Flow Diagram (CFD)**: A WIP area chart that also shows arrival rate (work items added daily), inventory (total work items in workflow), and lead time (time each item spends in system).
- **System Stability**: A system is stable when arrival rate and inventory remain constant over time; Kanban teams use **WIP limits to stabilize the system**.
- **Little's Law**: Applies to stable systems, stating that average lead time (W) is always equal to average inventory (L) divided by average arrival rate (λ) ($W = L \div \lambda$).
- **Reducing Lead Time**: If a workflow is stable with WIP limits, lead time can be reduced by stakeholders agreeing not to add new work items, which reduces the arrival rate.
- **Explicit Process Policies**: Kanban teams often make policies explicit by adding "definitions of done" or "exit criteria" to kanban board columns.
- **Emergent Behavior**: Gradual system improvement through WIP limits, flow management, and explicit policies often leads to improved behavior throughout the company.

Frequently Asked Questions: Impact of WIP Limits and Explicit Policies

- **Effectiveness**: Yes, WIP limits and explicit policies can significantly change team operations.
- **Origins**: Kanban and WIP limits are based on a work signaling system (named "Kanban" for "signal card" or "signboard") originating at **Toyota in the 1950s**.
    - In manufacturing, "kanbans" (physical cards) signal the need for more parts at assembly stations, smoothing out flow.

CHAPTER 10: The Agile Coach

1. Introduction to Agile Adoption Challenges

- Reading about agile values, principles, mindsets, and practices differs significantly from actually changing a team's way of working.
- Some teams can adopt Scrum or XP practices and immediately see great results if their mindset is already compatible with Agile Manifesto values and principles.
    - For such compatible teams, adopting agile feels easy because individuals don't need to change their thinking about work.
- **Barriers to Agile Adoption:** Challenges arise when:
    - The existing mindset is incompatible with Scrum, XP, or other agile methodologies.
    - The environment makes it difficult to succeed with agile values.

    - Individual contributions are rewarded far more than team effort.
    - Mistakes are punished harshly.
    - The environment stifles innovation.
    - The team lacks access to customers, users, or others who can help understand the software being built.

2. The Role of an Agile Coach

- An **agile coach** is someone who helps a team adopt agile.
- They assist each person on the team in learning a new attitude and mindset.
- Agile coaches help overcome mental, emotional, and technical barriers to agile adoption.
- They work with every team member to help them understand not just the "how" but also the "why" of new practices.
- A coach helps the team overcome the natural dislike and fear of change associated with trying new things at work.
- Teams need an **agile mindset** to achieve good results with an agile methodology.
    - Agile uses the values and principles of the Agile Manifesto to foster the right mindset.
    - Each agile methodology also has its own values and principles for the same reason.
    - The best results from agile adoption occur when each person's mindset is compatible with agile values, principles, and the specific methodology being adopted.
- **Goal of the Agile Coach:** To help the team attain a better, more agile mindset.
    - A good coach helps the team choose a methodology that best fits their existing mindset.
    - They introduce values, principles, and practices in a way that works for the team.
    - They help the team adopt practices and then use those practices to internalize values and principles, gradually changing attitudes and fostering the right mindset. This moves teams beyond merely "better-than-not-doing-it" results.
- This chapter covers how teams learn, how an agile coach facilitates mindset change for agile adoption, and how they help teams become more agile.

3. Case Study: Catherine, Timothy, and Dan

- **Scenario:** Catherine, a developer, and Timothy, another developer, work on a mobile phone camera app for a company bought by a large Internet conglomerate. Dan is their boss.
- Catherine and Timothy successfully used Kanban for eight months, improving their process.
    - Catherine identified bottlenecks, particularly managers (like Dan) dropping new features randomly.
    - Adding a queue led to managers stopping demands for more work than the team could physically handle.
- Dan informs Catherine that their parent company noticed their success and wants them to teach other teams.
- Catherine expresses unfamiliarity with coaching. Dan states it is not optional.

4. Coaches Understand Why People Don’t Always Want to Change

- Most people in an organization aim to do a good job and want to be seen as competent by peers and supervisors.
- When comfort and familiarity are established in a job, people resist adopting entirely new ways of doing things.
- Agile coaches spend most of their time helping people change how they work, which is challenging for both coach and team.
    - The coach sees the big picture, but team members are asked to adopt new rules without necessarily knowing why.
- **Ways Practices are Altered (without true agile adoption):**
    - **Daily Scrum becomes a status meeting:** Instead of fostering self-organization, it becomes a report meeting where the Scrum Master assigns work, resembling command-and-control project management.
    - **User stories added to BRDs:** Business Requirements Documents are still treated as specifications, leading to "big requirements up front" (BRUF) development, not true agile.
    - **Post-code test development (not TDD):** Teams ensure high code coverage after building the code, meaning tests don't impact design, which is already complete. This isn't true Test-Driven Development (TDD).
- In these cases, teams genuinely try to adopt agile but don't understand how practices fit into a larger methodology ecosystem.
    - They focus on familiar parts of practices instead of changing their work methods.
    - Teams accustomed to command-and-control lack experience with self-organization and context for the Daily Scrum beyond what they already know.
- Teams adopting agile often have some success already and naturally seek small, incremental changes to avoid breaking what works.
- **Biggest Roadblock to Agile Adoption:** When a team member thinks, "I've seen agile, I've adopted the familiar practices, my team is working better than before, and that's enough for me".
    - This leads to "better-than-not-doing-it" results, causing many to dismiss agile as just hype or marginal improvements.
- **Reasons for Rejecting Unfamiliar Practices:**
    - Every new practice is a change, and there's a risk it won't work out.
    - Failure at work can lead to job loss.
    - Coaching involves helping teams change, which can evoke an "outsized—but entirely rational—emotional response" because work provides for one's life and family.
    - Learning new things at work, if not mastered quickly, causes severe emotional discomfort. A basic primal instinct is fear of not being able to do one's job and provide. This can lead to anxiety and seemingly irrational reactions.
    - People gravitate towards what they understand because they often lack time to think through reasons for change.
    - **Pilot Project Pitfalls:** Adopting agile through a pilot project, often led by one person who read a book, while simultaneously dealing with deadlines, bugs, and changing requirements, is not ideal. Teams may keep agile labels (e.g., "Sprint" for milestones, unutilized task boards) but revert to old methods due to deadlines.

- When teams use agile names but don't change how they work, they become disillusioned, believing agile doesn't work, without ever truly experiencing it.
    - This happens because they are asked to change without understanding "why" and without help through the change that maintains the integrity of new practices.
- **Agile Coach's Crucial Job:** To recognize when people are asked to do something new and help them feel comfortable with the change.
    - The coach provides context for the change, explaining the "why" (not just "what"), which helps teams genuinely change, not just rename old practices.
    - **Examples of Coach Explanations:**
        - How the Daily Scrum helps teams self-organize.
        - How test-driven development supports functional thinking and incremental design.
        - How user stories help understand the user's perspective.
    - The coach helps teams go beyond adopting rules to see where they are going and what the new practices will eventually provide.

5. Warning Signs That the Team Is Having Trouble with a Change

- Agile coaches hear recurring sentiments from team members indicating discomfort with change.
    - **"We already build software well. Why are you telling me to do something different?"**
        - Teams with a history of success need a strong "why" for change; "the boss said so" undermines morale.
        - Coach's approach: Stay positive about past work, but point out problems they encountered. Agile practices aim to fix team problems; explaining "why" those problems happen and offering solutions increases acceptance.
    - **"This is far too risky."**
        - Common response, especially from those used to command-and-control project management. They seek buffers, risk registers, and bureaucracy for "CYA".
        - Opaque project plans with vague milestones and buffers provide comfort. Status reports control the message to users.
        - Agile feels risky because working software is the primary measure of progress, and problems are immediately apparent to everyone.
        - Coach's approach: Help managers, users, and customers get comfortable with this "undiluted" measure of progress. Create a safe environment where teams are allowed to fail today as long as they learn tomorrow. If not immediately realistic, help set a future goal to change the team's climate and attitude.
    - **"Pair programming (or test-driven development, or some other practice) just isn't going to work for me."**
        - A solo developer might feel pair programming slows them down. This feeling can be rational if mistakes are unacceptable in their environment, making it uncomfortable to be watched while coding.
        - A senior team member might be uncomfortable letting a junior control the keyboard.

        - Coach's approach: Help the team choose practices compatible with their current mindset. As practices are adopted, team members will see benefits and understand their mechanics, gradually shifting to a more agile mindset with coach guidance.
    - **"Agile doesn't work for a business like ours."**
        - People used to large, detailed plans and designs struggle to imagine other ways.
        - Command-and-control managers and architects find comfort in upfront scope, requirements, and design on paper. They are asked to trust the team to make decisions at the last responsible moment, which means giving up control.
        - They argue their business complexity necessitates extensive upfront planning.
        - Coach's approach: Explain that every business is complex and needs planning, but teams are better at capturing complexity by dividing projects into smaller pieces and having freedom to make decisions at the last responsible moment.
    - **"This is exactly like what we're already doing, just with a different name."**
        - One of the most common reasons for rejecting agile. Teams adopt new names for existing practices, making agile seem trivial or even wrong if the old practice failed.
        - This leads to people denouncing agile as "hype" or an "obfuscation".
        - Coach's approach: Recognize this isn't malicious, but a natural reaction from someone who thinks they've seen agile but haven't. The coach's job is to help the team understand that agile is genuinely different and effective.

6. Coaches Understand How People Learn: The Shu Ha Ri Model

- A good model for mastering anything comes from martial arts: Shu Ha Ri.
    - **Shu (守, "obey" or "observe"):** Follow the rule.
    - **Ha (破, "detach" or "break"):** Break the rule.
    - **Ri (離, "separate"):** Be the rule.
- Kent Beck (XP author) described XP's three levels of maturity similarly:

1. Do everything as written.
2. Experiment with variations after doing everything as written.
3. Eventually, don't care if you are doing XP or not.

- Different people initially perceive agile differently (like blind men and the elephant).
    - A programmer might see XP practices and conclude agile is about programming, design, and architecture.
    - A project manager might see Scrum practices and decide agile is about improving project management.
- Most people learn agile by doing, not by reading. They want simple, straightforward rules to follow for building better software faster.
- Coaches can be frustrated when teams focus on tangible practices (like task boards) instead of abstract values (like self-organization).
- **Core Coaching Principle:** "Meet them where they are, not where you want them to be".

    - A good agile coach understands how people learn and provides information, guidance, and examples to help them reach their next learning stage.
    - Coaches understand that people don't change overnight.
- **Shuhari Stages in Detail:**
    - **Shu (守, "obey" or "observe"):**
        - Mindset of someone new to an idea or methodology.
        - Wants simple rules to follow.
        - People latch onto practices (sprints, pair programming, task boards) because they can be made into rules everyone can follow.
        - Rules are important for adults learning new things at work, providing a starting point.
        - Simple practices in agile methodologies work well for those in the shu stage, allowing them to focus on getting good at them.
        - Setting a shu-level rule (e.g., "Daily Scrum at 10:30 where each person answers three questions") helps guide toward learning a principle (e.g., "self-organization to constantly review and adjust our plan").
        - **Warning:** A coach's goal is not to establish agile orthodoxy. An **agile zealot** focuses only on learned rules, stopping at the shu level, believing every problem has a known rule-based solution. Zealots make poor coaches because they don't understand where people are in their learning.
    - **Ha (破, "detach" or "break"):**
        - Stage where people have practiced the rules and begin to understand and internalize the principles driving them.
        - Effective way to change mindset is to adopt practices, and through them (and discussion), understand values and principles.
        - When a team reaches the ha stage together, they progress from "better-than-not-doing-it" results to true productivity gains from an agile mindset.
    - **Ri (離, "separate"):**
        - Stage of fluency in ideas, values, and principles.
        - Less concern about methodologies because the individual has gone beyond them.
        - When a problem arises, the team naturally applies the necessary practice, regardless of its methodology name.
        - Not chaos, but a smooth, well-functioning team where everyone "just does what's right".
        - Can sound "distressingly Zen". Phrases like "Do whatever works," "When you are really doing it, you are unaware that you are doing it," or "Use a technique so long as it is doing some good" are true for fluent learners but confusing or useless to those at lower stages.
- **Coach's First Job with Shuhari:** Figure out each person's current stage of learning.
    - A team with people at the ri stage is easy to work with because they know how to learn and adapt.
    - A person at the shu stage wants unambiguous rules. The coach should provide a rule even for trivial decisions (e.g., stickies vs. index cards for a task board). Later, the coach can explain the "why" and how different options fulfill the same principles.
- **Using Shuhari to Help Teams Learn Values:**

    - Shuhari explains why teams frustrated with abstract values only want to talk about tangible practices like task boards and stories.
    - Task boards and stories are easily presented as shu-level concepts (straightforward rules). Scrum has many shu-level rules, contributing to its successful adoption.
    - Values like openness and commitment are ha-level ideas: abstract concepts governing how rules are used.
    - Self-organization and collective commitment happen only when values are truly understood and internalized.
    - Shuhari helps understand that teams need to go through the motions of practices before they can "detach" and grasp the deeper meaning of values.
    - Adopting Scrum practices is an effective first step to learning its values.
    - A good coach is patient, using opportunities during sprints to teach values. These opportunities often arise when different perspectives create a "fractured perspective".
    - **Example (Scrum Feature Conflict):**
        - **Scenario:** Developer realizes a feature won't be done, asks Product Owner to push it. Product Owner gets angry, wants it in ("do whatever it takes"), even if corners are cut.
        - **Shu-level Answer:** Scrum rules state a feature must be "done done" for demo, incomplete work is pushed.
        - **Ha-level Teaching Opportunity:** Coach helps Product Owner see the developer's perspective. Poor work adds technical debt, making code harder to maintain and slowing future sprints. Ignoring the rule might get the feature sooner, but future development will be slower, teaching the Product Owner about **Scrum's value of focus** (completing work for higher quality, faster delivery).
        - Coach also helps the developer understand the feature's importance and value to customers. This helps the developer learn about **Scrum's value of commitment** (effective teams understand customer value) and how to break features into smaller pieces for faster value delivery.
- A coach with deep methodology understanding fills a valuable role, helping teams adopt practices and rules, and through those rules, understand agile values to foster an agile mindset.

7. Coaches Understand What Makes a Methodology Work

- Conflicts and problems are daily occurrences in projects; not all present teaching opportunities about Scrum values.
- **Recognizing Teaching Opportunities:** A coach recognizes opportunities because they understand not just *how* Scrum teams use iteration, but *why*.
    - A timeboxed iteration becomes ineffective if incomplete work is allowed. Without the "done done" rule, iterations devolve into simple project milestones, with variable scope and no requirement to deliver working software.
- This breakdown violates important agile values and principles:
    - Working software is the primary measure of progress; incomplete software gives a false measure.

    - Agile teams value customer collaboration, but this doesn't mean the customer is always right. It means collaborating to find solutions that deliver the most value if a plan problem is recognized.
    - If customers/Product Owners can pressure developers to include incomplete software, the team becomes guarded, and collaboration gives way to contract negotiation.
- A coach understands values and principles of agile and specific methodologies (Scrum, XP, Kanban) and how they drive practices. They recognize what makes methodologies "tick".
    - Scrum coach understands collective commitment and self-organization.
    - XP coach understands embracing change and incremental design.
    - Kanban coach understands WIP limits for flow control; preventing WIP limits from being set unravels the methodology.
- **Let the Team Fail (with Caution):**
    - Lyssa Adkins suggests coaches allow teams to fail, but not "careen off a cliff".
    - Failing and recovering together strengthens and speeds up teams.
    - Sometimes, what seems harmful may work out. Coaches should "watch and wait".
    - This counters "command-and-control-ism" in coaches. A good coach knows when failure is the most effective path to project and team success.
    - **Retrospectives** are valuable here. They allow teams to learn from failure.
        - If failure is due to poor practice implementation, the coach teaches skills to improve.
        - If failure is due to mindset, the coach uses it to teach the specific value or principle that would have led to better decisions. This is how teams grow.
- **When Not to Allow Failure:**
    - A good coach understands that some "supporting beams" (core elements) of a methodology cannot be changed without compromising integrity.
    - A coach needs a **ha-level understanding** of the methodology as a system, knowing *why* it works.
    - **Example:** A Kanban boss removing WIP limits (seen in Chapter 8) can unravel the entire Kanban effort. A shu-level understanding might allow it, but a good Kanban coach knows WIP limits are the "lynchpin" holding the methodology together.
- **Explaining Complexity:** One difficult part of coaching is determining how much to explain to the team, boss, and customers.
    - A simple shu-level explanation of rules gets the job done but doesn't lead to true agile mindset.
    - Teams might feel they've just traded one set of arbitrary rules for another.
    - A ha-level understanding helps them see *why* rules produce better software, but it can sound "distressingly Zen".
    - The coach's role is to help the team move toward a ha-level understanding at their own pace, avoiding overly abstract explanations.

8. The Principles of Coaching

- Agile requires the right mindset, achieved through values and principles; hence, methodologies have values that teams internalize for full potential.

- Coaching also has its own set of values. John Wooden, a renowned basketball coach, laid out five basic principles of coaching that apply to an agile coach:
    - **Industriousness:**
        - Changing how a team builds software requires hard work on new things.
        - Developers learn planning and testing; Product Owners understand team work; Scrum Masters learn to give control while staying involved. These are new skills requiring effort.
    - **Enthusiasm:**
        - Excitement about a new way of working is contagious.
        - Enthusiasm comes from solving past problems and leads to more innovative, happier, and collaborative teams.
    - **Condition:**
        - Agile works when every team member excels at their job.
        - There's an implicit assumption of **pride of workmanship**: striving to build the best software and continuously improving skills.
        - Team members maintain their "best condition" by working hard to improve skills.
    - **Fundamentals:**
        - "The finest system cannot overcome poor execution of the fundamentals". Coaches must avoid getting "carried away" by complicated systems.
        - Agile works due to its simple, straightforward values and uncomplicated practices.
        - A good coach keeps the team focused on these fundamentals and helps keep things simple.
    - **Development of Team Spirit:**
        - Agile encourages self-organization, whole team collaboration, energized work, and empowering the team.
        - An agile coach must identify and address team members focused primarily on personal performance, career goals, or getting promoted out of the team.
        - Selfishness, envy, egotism, and criticism destroy team spirit.
        - The coach must actively encourage teamwork and unselfishness, and be vigilant to prevent negative traits at the source. Team members must be eager to sacrifice personal glory for the team's welfare.
- These five principles form a solid foundation for an agile coach's mindset and should be understood, internalized, and used like the values of the Agile Manifesto and any agile methodology.

9. Key Points (Summary from Source)

- An agile coach helps a team adopt agile and helps each individual learn a new mindset.
- Effective agile coaches help teams overcome discomfort with new practices by focusing on familiar aspects.
- Coaches recognize warning signs that teams are uncomfortable with change.
- People learn new systems or mindsets in three stages: **Shuhari**.
- In the first stage, **shu**, learners are new to an idea and best respond to specific instructions/rules.

- The **ha** stage is when someone recognizes the larger system and adapts their mindset to match it.
- In the **ri** stage, people are fluent with the system's ideas and know when rules don't have to be followed.
- An effective agile coach knows *why* a system works and understands the purpose of its rules.